

Graph Mastery — Java Edition

9 patterns. 36 problems. Pattern-first FAANG-prep cheatsheet — every template in Java.

Java translation of the original "Graph Mastery" sheet by @rathoreadiitya (Instagram). v1.0-java · 2026-06-07

Why this sheet exists

Graph is the topic that decides FAANG interviews. Every coding round above "screening" has a graph problem, and most system-design rounds have graph thinking baked in. Students burn weekends on it with little to show because they treat it as "dozens of algorithms" instead of 9 patterns.

This sheet collapses graph into 9 patterns: DFS, BFS, Topological Sort, Union-Find, Dijkstra, Bellman-Ford/Floyd-Warshall, MST, Grid-as-graph, Bipartite check. Each pattern has a mental model, Java code walkthrough, traps, warm-up, and 3-4 LeetCode problems in the exact order to solve them. Own these 9 patterns cold and you cover roughly 90% of graph problems FAANG asks.

This edition is Java-first. Every code block is idiomatic Java, and every language-specific note (recursion depth, heaps, hash maps, overflow) is written for the JVM — not Python. Where the original said `sys.setrecursionlimit`, this one tells you how Java's call stack behaves and what to do instead.

How to use: Crash course → 5 min. Decision tree → internalise it. Patterns 1-9 in order — each has a warm-up, walkthrough(s), traps, and curated practice. Cross-pattern traps → read once before any interview. Revision drill → 20 minutes the night before.

Inside this sheet

1. Crash course (5 min)
 2. Pattern decision tree (problem → pattern in 10 seconds)
 3. Java toolkit (Python → Java cheat map)
 4. Pattern 1 — DFS
 5. Pattern 2 — BFS
 6. Pattern 3 — Topological sort
 7. Pattern 4 — Union-Find (DSU)
 8. Pattern 5 — Dijkstra (shortest path, non-negative)
 9. Pattern 6 — Bellman-Ford & Floyd-Warshall
 10. Pattern 7 — MST (Kruskal + Prim)
 11. Pattern 8 — Grid as graph
 12. Pattern 9 — Bipartite (2-coloring)
 13. Cross-pattern traps (read once before any interview)
 14. Pre-interview revision drill (20 min)
 15. 7-day study plan
-

Crash course (5 min)

A graph = (V, E): vertices V and edges E (directed or undirected, weighted or not). Almost every graph problem reduces to one of these 6 questions:

- "Can I reach X from Y?" / "how many components?" → DFS or BFS (Patterns 1, 2).
- "Shortest path in an UNWEIGHTED graph." → BFS (Pattern 2).
- "Shortest path in a WEIGHTED graph (non-negative)." → Dijkstra (Pattern 5).
- "Shortest path WITH negative weights / all-pairs." → Bellman-Ford / Floyd-Warshall (Pattern 6).
- "Order tasks with dependencies / detect cycle in a DAG." → Topo sort (Pattern 3).
- "Connect all nodes at minimum cost / dynamic connectivity." → MST + DSU (Patterns 4, 7).
- "Can I 2-color the graph / split nodes into two opposing groups?" → Bipartite check (Pattern 9).

Plus one structural insight:

- A grid IS a graph (Pattern 8). Don't build an adjacency list — use direction deltas.

The big insight you keep missing: "Graph algorithm" doesn't mean a different algorithm per problem. Most problems are DFS / BFS in disguise. Reach for the heavyweights (Dijkstra, MST, Floyd) only when the question literally says weighted, negative, all-pairs, or spanning.

Pattern decision tree (problem → pattern in 10 seconds)

What the problem says	Pattern
"Can I reach" / "number of components / islands / provinces"	1 — DFS
"Flood fill" / "mark a connected region" / "path exists"	1 — DFS
"Shortest path in unweighted graph" / "min moves / steps"	2 — BFS
"Multi-source" / "rotting oranges" / "01 matrix"	2 — BFS (multi-source)
"Course schedule" / "build order" / "dependencies" / "X before Y"	3 — Topological sort
"Dynamic connectivity" / "accounts merge" / "redundant connection"	4 — Union-Find
"Shortest path, WEIGHTED, non-negative" / "network delay" / "min effort"	5 — Dijkstra
"Negative weights" / "detect negative cycle" / "K-stop limit"	6 — Bellman-Ford
"All-pairs shortest path" / " $V \leq \sim 400$ " / "evaluate division"	6 — Floyd-Warshall
"Min cost to connect ALL nodes" / "spanning tree"	7 — MST (Kruskal)
"Island / matrix / grid + path / fill"	8 — Grid as graph
"2-color the graph" / "possible bipartition" / "two teams / sides"	9 — Bipartite check

Java toolkit (Python → Java cheat map)

The original sheet is in Python. Here is the one-to-one translation you will reuse in every pattern below.

Python	Java	Notes
collections.deque	ArrayDeque<>	BFS queue: offer / poll. Stack: push / pop. Never null elements.
heapq (min-heap of tuples)	PriorityQueue<int[]>	Pass a comparator; default is min-heap. No decrease-key — push & skip stale.

defaultdict(list)	Map<Integer,List<Integer>> + computeIfAbsent, or List<List<Integer>>	For 0..n-1 nodes, prefer an array/list of lists.
tuple (a, b, c)	int[]{a, b, c} or a small record	Arrays are fastest; records are readable.
float('inf')	Integer.MAX_VALUE / Long.MAX_VALUE / sentinel 1_000_000_000	Watch overflow when you add to it (see Dijkstra/Floyd).
sys.setrecursionlimit(10 ⁶)	*no equivalent*	JVM throws StackOverflowError. Go iterative, or run DFS on a big-stack thread (below).
list comprehension	for loop / Stream	Loops are clearest for graph traversal.
grid[r][c] = '#'	same, on char[][]	In-place visited marking works identically.

Recursion depth in Java (the question the original answers for Python): Java has no setrecursionlimit. The recursion ceiling is the **thread stack size** — by default roughly 512 KB, i.e. ~10k-15k frames. When a deep graph might overflow it (e.g. a 200×200 grid that degenerates into a 40,000-cell snake), do one of:

```
// Option A – go iterative with an explicit stack (preferred, no surprises)
Deque<int[]> stack = new ArrayDeque<>();
stack.push(new int[]{startR, startC});
while (!stack.isEmpty()) {
    int[] cell = stack.pop();
    // ... visit, push unvisited neighbours
}

// Option B – run recursive DFS on a thread with a large stack (e.g. 64 MB)
Thread t = new Thread(null, () -> solve(), "dfs", 1 << 26);
t.start();
t.join();
```

Comparator overflow: (a, b) -> a[0] - b[0] overflows if values can be large/negative. Prefer Integer.compare(a[0], b[0]) for any heap that may hold big distances.

Pattern 1 — DFS — Depth-First Search (connectivity, components, ordering)

One-liner: Go DEEP first, mark visited, backtrack. Use recursion (or an explicit Deque stack) to enumerate reachable nodes, count components, detect cycles, or build an order.

Mental model

DFS is recursion with a visited set. Visit a node, mark it, recurse into every unvisited neighbour, return. That single skeleton solves connectivity, components, flood fill, path existence, cycle detection (directed: via the recursion stack), and post-order topological sort.

Invariant: Once dfs(u) returns, every node reachable from u has been visited and marked. Re-entering a visited node is a no-op.

Recursive vs iterative in Java: Recursive DFS is shorter and is the right default for typical LeetCode sizes. Unlike Python you cannot raise a frame limit — the JVM throws StackOverflowError once the thread stack (~10k-15k frames) is exhausted. If worst-case depth could exceed ~10⁴, convert to an explicit Deque<int[]> stack or run on a big-stack thread (see the Java toolkit above). Default to

recursion; reach for the iterative form only when depth is a real risk.

Spot signals

- "Reach all nodes from X" / "is there a path from A to B?"
- "Count connected components" / "number of provinces / islands"
- "Flood fill" / "colour a region" / "surround captured regions"
- "Cycle in a DIRECTED graph" (use the recursion-stack / 3-colour flag, NOT just visited).
- "All paths from source to target" — DFS with backtracking emits each path.

Walkthrough — LC 200 Number of Islands (DFS flood fill)

Sweep the grid; every time you find a '1' that isn't visited, DFS to sink the whole island and increment the counter. Direction deltas avoid building an adjacency list.

```
private static final int[][] DIRS = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};

public int numIslands(char[][] grid) {
    if (grid == null || grid.length == 0) return 0;
    int R = grid.length, C = grid[0].length, count = 0;
    for (int r = 0; r < R; r++) {
        for (int c = 0; c < C; c++) {
            if (grid[r][c] == '1') {
                dfs(grid, r, c, R, C);
                count++;
            }
        }
    }
    return count;
}

private void dfs(char[][] grid, int r, int c, int R, int C) {
    if (r < 0 || r >= R || c < 0 || c >= C) return;
    if (grid[r][c] != '1') return;
    grid[r][c] = '#'; // mark visited in place
    for (int[] d : DIRS) dfs(grid, r + d[0], c + d[1], R, C);
}
```

Walkthrough — cycle in a DIRECTED graph (DFS 3-colour)

WHITE = unvisited, GRAY = on the current DFS path, BLACK = finished. A back-edge to a GRAY node is a cycle.

```

private static final int WHITE = 0, GRAY = 1, BLACK = 2;

public boolean hasCycle(int n, List<List<Integer>> graph) {
    int[] color = new int[n];           // all WHITE (0) by default
    for (int i = 0; i < n; i++)
        if (color[i] == WHITE && dfs(i, graph, color)) return true;
    return false;
}

private boolean dfs(int u, List<List<Integer>> graph, int[] color) {
    color[u] = GRAY;
    for (int v : graph.get(u)) {
        if (color[v] == GRAY) return true;           // back-edge -> cycle
        if (color[v] == WHITE && dfs(v, graph, color)) return true;
    }
    color[u] = BLACK;
    return false;
}

```

Common traps

- **Marking visited AFTER recursing instead of BEFORE.** The same node gets pushed many times — exponential blowup. Mark `grid[r][c] = '#'` (or `visited[u] = true`) **before** you recurse.
- **Using a single `boolean[]` visited for cycle detection in a DIRECTED graph.** `visited[v] = true` can't tell "seen in a different DFS" from "on my current path." Use 3-colour or an `onStack[]` flag.
- **Stack depth.** Java has no `setrecursionlimit`; deep graphs throw `StackOverflowError`. Go iterative or use a big-stack thread.
- **Mutating the grid for visited tracking when the problem says "do not modify input."** Use a separate `boolean[][]` visited instead.

Warm-up first — build the mechanics

Solve this cold before LC 200. LC 733 is the simplest flood-fill: one starting cell, one colour, no counting. It locks the "bounds check + visited mark + recurse 4 directions" rhythm before you add the component-counting loop.

#	Problem	Difficulty
733	Flood Fill — single-source DFS recoloring; the simplest grid-DFS problem	Easy

Practice (solve in this order — each unlocks the next)

Step	#	Problem	Difficulty
1	200	Number of Islands — the canonical DFS / flood fill; bar-raiser foundation	Medium
2	695	Max Area of Island — DFS returning a size; trivial extension of LC 200	Medium
3	841	Keys and Rooms — DFS reachability on an adjacency list; simplest non-grid DFS	Medium
4	130	Surrounded Regions — reverse-flood-fill from the border; "mark survivors first"	Medium
5	133	Clone Graph — DFS with a node→copy HashMap; canonical "deep-copy a graph"	Medium

Pattern 2 — BFS — Breadth-First Search (shortest path, level order, multi-source)

One-liner: Process nodes in WAVES from the source(s). The queue is the data structure. For unweighted (or equal-weighted) graphs, the first time you reach a node is via the shortest path.

Mental model

BFS is DFS's level-by-level cousin. Mark visited when you ENQUEUE (not when you dequeue) to avoid pushing the same node many times. Track distance by enqueueing `int[] {node, dist}` or by processing the queue in level-sized chunks (`int sz = q.size(); for (int i = 0; i < sz; i++) ...`).

Multi-source BFS: When the question is "min distance from ANY source to each cell," enqueue ALL sources with dist 0 BEFORE the loop. The waves expand from all of them at once — same $O(V+E)$, no per-source restart.

Invariant: When node `u` is first dequeued, `dist[u]` is the minimum number of edges from the (closest) source to `u`.

Spot signals

- "Shortest path in an UNWEIGHTED graph" / "minimum steps / moves to reach"
- "Level order traversal" (a tree is just a graph with no cycles)
- "Word Ladder" / "generate-all-1-edit-neighbours"
- "Rotting oranges" / "01 matrix" / "walls and gates" — multi-source BFS giveaway.
- "Shortest bridge between two islands" (BFS from the entire boundary of one island).

Walkthrough — LC 994 Rotting Oranges (multi-source BFS)

Enqueue EVERY rotten orange first with time = 0, then BFS. The answer is the last timestamp emitted (or -1 if any fresh oranges remain).

```

public int orangesRotting(int[][] grid) {
    int R = grid.length, C = grid[0].length, fresh = 0;
    Deque<int[]> q = new ArrayDeque<>();
    for (int r = 0; r < R; r++) {
        for (int c = 0; c < C; c++) {
            if (grid[r][c] == 2) q.offer(new int[]{r, c, 0}); // all sources first
            else if (grid[r][c] == 1) fresh++;
        }
    }
    int[][] dirs = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
    int time = 0;
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        int r = cur[0], c = cur[1], t = cur[2];
        time = t;
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] == 1) {
                grid[nr][nc] = 2; // mark on enqueue
                fresh--;
                q.offer(new int[]{nr, nc, t + 1});
            }
        }
    }
    return fresh == 0 ? time : -1;
}

```

Common traps

- **Marking visited on DEQUEUE.** The same node gets enqueued many times before processing — quadratic blowup. Mark on ENQUEUE.
- **Using BFS for shortest path on a WEIGHTED graph.** BFS only works for unweighted (or 0/1 edge weights — 0-1 BFS with a deque). For arbitrary weights use Dijkstra (Pattern 5).
- **Forgetting to track level/distance.** If you need the shortest distance, store `int[]{node, dist}` or process the queue level-by-level (`int sz = q.size(); for (int i = 0; i < sz; i++) ...`).
- **Word Ladder: building the full adjacency list ($O(N^2)$ words).** Build a bucket-keyed neighbour map (`"h*t" -> [hot, hit, hat]`) with a `HashMap<String, List<String>>` for $O(N \cdot L)$ generation instead.

Warm-up first — build the mechanics

Solve this cold before LC 994. LC 1971 is the simplest BFS / DFS connectivity check — one source, one sink, yes/no. It locks the queue-marker-loop rhythm without grid coordinates and time tracking on top.

#	Problem	Difficulty
1971	Find if Path Exists in Graph — single-source reachability BFS; simplest BFS template	Easy

Practice (solve in this order — each unlocks the next)

Step	#	Problem	Difficulty
1	994	Rotting Oranges — the canonical multi-source BFS; bar-raiser pattern	Medium
2	102	Binary Tree Level Order Traversal — BFS on a tree; the level-by-level skeleton	Medium

3	127	Word Ladder — BFS with on-the-fly neighbour generation via bucket map	Hard
4	542	01 Matrix — multi-source BFS from all 0s outward; elegant alternative to 2-pass DP	Medium

Pattern 3 — Topological sort (Kahn's BFS / DFS post-order)

One-liner: Order vertices of a DAG so every edge $u \rightarrow v$ has u BEFORE v . Either remove zero-in-degree nodes (Kahn) or output DFS finish-order reversed.

Mental model

Topo sort is the canonical "dependency resolution" pattern — build order, course schedule, package install order, task dependencies.

Kahn's BFS: Compute the in-degree of every node. Enqueue all in-degree-0 nodes. Pop one, append to output, decrement each neighbour's in-degree, enqueue any that hit 0. If output length $< V$ at the end $\rightarrow$ the graph has a cycle.

DFS variant: Run DFS; when $\text{dfs}(u)$ FINISHES (post-order), push u onto a stack. Reverse the stack at the end. Cycle detection needs the 3-colour trick from Pattern 1.

Invariant (Kahn): At every step the queue contains exactly the nodes whose dependencies have all been emitted.

Spot signals

- "Course Schedule" / "build order" / "task dependencies"
- "Alien Dictionary" / "character precedence from sorted words"
- "Parallel courses" / "minimum semesters" (BFS level count = number of rounds)
- Any "X must come before Y" constraint plus "return a valid order or detect impossibility."

Walkthrough — LC 210 Course Schedule II (Kahn's BFS)

Build the graph + in-degree, BFS from in-degree-0 nodes, emit each and decrement neighbours. If the final order length $< \text{numCourses}$, there's a cycle $\rightarrow$ return an empty array.


```

public int[] findOrder(int numCourses, int[][] prerequisites) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
    int[] indeg = new int[numCourses];
    for (int[] p : prerequisites) {
        int course = p[0], prereq = p[1];
        graph.get(prereq).add(course);           // prereq -> course
        indeg[course]++;
    }
    Deque<Integer> q = new ArrayDeque<>();
    for (int i = 0; i < numCourses; i++) if (indeg[i] == 0) q.offer(i);
    int[] order = new int[numCourses];
    int idx = 0;
    while (!q.isEmpty()) {
        int u = q.poll();
        order[idx++] = u;
        for (int v : graph.get(u))
            if (--indeg[v] == 0) q.offer(v);
    }
    return idx == numCourses ? order : new int[0];
}

```

Common traps

- **Building the graph in the wrong direction.** [course, prereq] means prereq → course — NOT course → prereq. Reading it backwards inverts the whole algorithm.
- **Forgetting cycle detection.** If idx < numCourses at the end of Kahn's, the graph has a cycle. Return new int[0] (or whatever the problem requires).
- **Using DFS without the 3-colour trick.** A plain boolean[] visited can't tell a back-edge from a cross-edge — you'll miss cycles in DAGs that share dependencies.
- **Alien Dictionary edge case:** equal prefixes where the longer word comes first ("abc", "ab") is invalid — return "" immediately.

Warm-up note

Topo sort has no truly Easy LeetCode warm-up — every problem assumes the DAG framing. The honest warm-up: solve LC 207 Course Schedule (yes/no) cold BEFORE LC 210 (return the order). 207 just needs cycle detection; 210 adds output collection on top.

Practice (solve in this order — each unlocks the next)

Step	#	Problem	Difficulty
1	207	Course Schedule — the cycle-detection version (yes/no); the warm-up for 210	Medium
2	210	Course Schedule II — return the actual order; canonical Kahn's BFS template	Medium
3	802	Find Eventual Safe States — reverse-graph topo sort; nodes whose all paths terminate	Medium
4	310	Minimum Height Trees — trim leaves Kahn-style until ≤ 2 centroids remain	Medium

Pattern 4 — Union-Find (DSU) — connectivity in near-constant time

One-liner: Maintain a forest of disjoint sets with two ops: `find(x)` returns `x`'s group representative, `union(a, b)` merges two groups. With path compression + union by rank, each op is effectively $O(1)$.

Mental model

DSU is the right tool whenever you're streaming "merge these two groups" / "are these two in the same group?" queries — dynamic connectivity, equivalence classes, Kruskal's MST, cycle detection in undirected graphs (`find(u) == find(v)` BEFORE union $\rightarrow$ cycle).

Path compression: During find, point every node on the path directly to the root. Future finds are $O(1)$.

Union by rank: Attach the shorter tree under the taller one's root. Keeps trees shallow. Combined with compression, amortised cost is $\alpha(n)$ (inverse Ackermann) — essentially constant for any n you'd ever see.

Invariant: `find(x)` is the unique representative of `x`'s connected component. Two nodes are in the same component iff their finds match.

Spot signals

- "Number of connected components" in a streaming / merging setting
- "Redundant connection" / "remove an edge to make a tree"
- "Accounts merge" / "smallest equivalent string" / "equations satisfiability"
- "Kruskal's MST" (DSU + sorted edges = $O(E \log E)$)
- Anything where you'd be tempted to recompute components after every edge insert.

DSU class — path compression + union by rank

Memorise this class. Every DSU problem reuses it verbatim. (In Python it's ~14 lines; Java is a touch longer but mechanically identical.)

```

class DSU {
    int[] parent, rank;

    DSU(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];    // path compression (halving)
            x = parent[x];
        }
        return x;
    }

    boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;          // already merged
        if (rank[ra] < rank[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        if (rank[ra] == rank[rb]) rank[ra]++;
        return true;
    }
}

```

Walkthrough — LC 684 Redundant Connection

Iterate edges in input order; union each. The FIRST edge whose endpoints are already connected is the redundant one.

```

public int[] findRedundantConnection(int[][] edges) {
    DSU dsu = new DSU(edges.length + 1);    // nodes are 1-indexed
    for (int[] e : edges)
        if (!dsu.union(e[0], e[1])) return e;
    return new int[0];
}

```

Common traps

- **Forgetting path compression OR union by rank.** Without BOTH, find degrades to $O(\log n)$ or worse. The class above is the floor — nothing less in interviews.
- **Sizing the parent array wrong.** If nodes are 1-indexed (like LC 684), allocate $n + 1$ slots.
- **Using DSU for directed connectivity.** DSU only knows undirected. For directed strongly-connected questions use Kosaraju / Tarjan SCC.
- **Resetting DSU inside a loop that should accumulate.** Build it once; only union, don't reset between queries.

Warm-up note

DSU has no Easy LeetCode warm-up — the data structure itself is the lift. Memorise the class above cold BEFORE LC 684 / 547. If it doesn't flow out of your fingers, every DSU problem becomes a debugging session.

Practice (solve in this order — each unlocks the next)

Step	#	Problem	Difficulty
1	547	Number of Provinces — simplest DSU; union connected pairs, count distinct finds	Medium
2	684	Redundant Connection — detect the first edge that closes a cycle; iconic DSU	Medium
3	990	Satisfiability of Equality Equations — union all ==, then check any !=; two-pass	Medium
4	721	Accounts Merge — DSU on emails; the messiest real-world DSU; Amazon bar-raiser	Medium
5	947	Most Stones Removed — union stones sharing a row/col; answer = stones — components	Medium

Pattern 5 — Dijkstra — shortest path with NON-NEGATIVE weights

One-liner: Greedy: repeatedly expand the unvisited node with the SMALLEST known distance, relax its outgoing edges. Min-heap of {dist, node}. $O((V + E) \log V)$.

Mental model

Dijkstra is BFS where the queue is replaced by a min-heap keyed on distance. The next node to settle is always the one with the smallest tentative distance — because all weights are non-negative, no later expansion can give it a shorter path.

Lazy deletion: Java's PriorityQueue has no decrease-key, so push new (better) entries and skip stale ones with if ($d > \text{dist}[u]$) continue;.

Invariant: When a node is popped with $d == \text{dist}[u]$, $\text{dist}[u]$ is final — the shortest possible from the source.

Why non-negative only: Negative edges break the "first pop is final" guarantee — a later detour through a negative edge could improve it. Use Bellman-Ford (Pattern 6) for negatives.

Spot signals

- "Shortest path in a WEIGHTED graph with non-negative weights"
- "Network Delay Time" / "min time for signal to reach all nodes"
- "Path with Minimum Effort" / "Swim in Rising Water" — grid Dijkstra with custom cost.
- "Cheapest Flights" with a stop limit — Dijkstra variant (or Bellman-Ford).
- "Min cost to reach destination through a maze with weighted moves."

Walkthrough — LC 743 Network Delay Time

Classic Dijkstra. $\text{dist}[k] = 0$, all others infinity. Min-heap, pop smallest, relax outgoing.

```

public int networkDelayTime(int[][] times, int n, int k) {
    List<int[]>[] graph = new List[n + 1];
    for (int i = 1; i <= n; i++) graph[i] = new ArrayList<>();
    for (int[] t : times) graph[t[0]].add(new int[]{t[1], t[2]}); // {to, weight}

    int[] dist = new int[n + 1];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[k] = 0;

    // min-heap of {dist, node}; compare on dist (avoid a-b overflow)
    PriorityQueue<int[]> heap = new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));
    heap.offer(new int[]{0, k});

    while (!heap.isEmpty()) {
        int[] top = heap.poll();
        int d = top[0], u = top[1];
        if (d > dist[u]) continue; // stale entry
        for (int[] e : graph[u]) {
            int v = e[0], w = e[1];
            if (d + w < dist[v]) { // d == dist[u], always finite here
                dist[v] = d + w;
                heap.offer(new int[]{dist[v], v});
            }
        }
    }
    int ans = 0;
    for (int i = 1; i <= n; i++) ans = Math.max(ans, dist[i]);
    return ans == Integer.MAX_VALUE ? -1 : ans;
}

```

Common traps

- **Forgetting the stale-entry skip (if $d > \text{dist}[u]$ continue;).** Without it you re-relax the same node many times — still correct but $O(E^2 \log V)$ in pathological cases.
- **Running Dijkstra on negative edges.** Silently wrong — relaxation gives a non-optimal answer. Confirm non-negative before reaching for the heap.
- **Integer overflow.** Don't relax from a node still at `Integer.MAX_VALUE` (infinity + w overflows to negative). Relaxing only from popped nodes — whose `dist` is finite — avoids it; otherwise guard with `dist[u] != Integer.MAX_VALUE` or use long.
- **Comparator $a[0] - b[0]$.** Overflows with large distances. Use `Integer.compare(...)`.
- **Setting `dist[start] = 0` but NOT pushing $\{0, \text{start}\}$ onto the heap.** The loop relies on the source being popped first.
- **Using Dijkstra for unweighted graphs.** BFS is simpler and equivalent — heap overhead is wasted.

Warm-up note

Dijkstra has no Easy LeetCode warm-up — it's Medium-floor. The honest warm-up: solve LC 1971 (BFS, Easy) to lock the "source + frontier + mark + expand" rhythm WITHOUT weights, then swap the queue for a `PriorityQueue` and add the relaxation step — that's all Dijkstra is.

Practice (solve in this order — each unlocks the next)

Step	#	Problem	Difficulty
------	---	---------	------------

1	743	Network Delay Time — textbook Dijkstra; mandatory FAANG problem	Medium
2	1631	Path with Minimum Effort — Dijkstra on a grid with max-edge-along-path cost	Medium
3	778	Swim in Rising Water — Dijkstra OR binary-search + BFS; the bar-raiser version	Hard
4	1514	Path with Maximum Probability — Dijkstra with multiply + max-heap (inverted relaxation)	Medium

Pattern 6 — Bellman-Ford & Floyd-Warshall (negatives + all-pairs)

One-liner: When weights can be NEGATIVE, Dijkstra fails. Bellman-Ford handles negatives in $O(V \cdot E)$; Floyd-Warshall handles ALL pairs in $O(V^3)$ and is the cleanest code on the entire sheet.

Mental model

Bellman-Ford: Relax EVERY edge $V-1$ times. After that, all shortest paths are settled (any shortest path uses $\leq V-1$ edges). One MORE pass that still improves anything $\rightarrow$ there's a negative cycle reachable from the source.

Floyd-Warshall: DP over intermediate vertices. $\text{dist}[i][j] = \min \text{ over } k \text{ of } \text{dist}[i][k] + \text{dist}[k][j]$. The key detail: k MUST be the OUTERMOST loop — each iteration says "now vertex k is also allowed as an intermediate." Inverting the loops gives nonsense.

When to use which: Bellman-Ford = single-source with possible negatives. Floyd-Warshall = all-pairs OR very small V ($\leq \sim 400$) where $O(V^3)$ is fine.

Bellman-Ford bonus: Limiting passes to $K+1$ gives "shortest path with AT MOST K edges" — the trick behind LC 787.

Spot signals

- "Shortest path with NEGATIVE weights" / "detect negative cycle"
- "All-pairs shortest path" / "from every city to every other"
- "Cheapest Flights Within K Stops" — limited-edge variant of Bellman-Ford
- Small V ($\leq$ a few hundred) with multiple sources $\rightarrow$ Floyd-Warshall beats running V Dijkstras
- "Transitive closure" — Floyd-Warshall with OR/AND replacing min/+.

Walkthrough — Bellman-Ford

$V-1$ relaxation rounds, then one detection round. Returns null on a negative cycle. Uses long so adding to "infinity" can't overflow.

```

public long[] bellmanFord(int n, int[][] edges, int src) {
    long[] dist = new long[n];
    Arrays.fill(dist, Long.MAX_VALUE);
    dist[src] = 0;
    for (int i = 0; i < n - 1; i++) {
        for (int[] e : edges) {
            int u = e[0], v = e[1], w = e[2];
            if (dist[u] != Long.MAX_VALUE && dist[u] + w < dist[v])
                dist[v] = dist[u] + w;
        }
    }
    for (int[] e : edges) { // detection pass
        int u = e[0], v = e[1], w = e[2];
        if (dist[u] != Long.MAX_VALUE && dist[u] + w < dist[v])
            return null; // negative cycle reachable
    }
    return dist;
}

```

Walkthrough — Floyd-Warshall (the k-loop is OUTERMOST)

That ordering is the entire correctness proof. Use a large sentinel (1e9), not Integer.MAX_VALUE, so $\text{dist}[i][k] + \text{dist}[k][j]$ can't overflow.

```

public int[][] floydWarshall(int n, int[][] edges) {
    final int INF = 1_000_000_000;
    int[][] dist = new int[n][n];
    for (int[] row : dist) Arrays.fill(row, INF);
    for (int i = 0; i < n; i++) dist[i][i] = 0;
    for (int[] e : edges) dist[e[0]][e[1]] = e[2]; // directed; add reverse if undirected

    for (int k = 0; k < n; k++) // k MUST be outermost
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];

    return dist;
}

```

Common traps

- **Floyd-Warshall with k NOT outermost.** Silently wrong — gives a non-optimal matrix. Memorise the order: k, then i, then j.
- **Bellman-Ford with V passes instead of V-1.** Not wrong, just wastes a round. The detection pass is the Vth.
- **Overflow.** With a MAX_VALUE sentinel, $\text{INF} + w$ wraps negative and corrupts the answer. Use long (Bellman-Ford) or a 1e9 sentinel (Floyd) and guard before adding.
- **Reaching for Dijkstra when negative weights are present.** Always inspect the weight range first.
- **Using Floyd-Warshall for $V > \sim 500$.** $O(V^3)$ time and $O(V^2)$ memory blow up fast.
- **LC 787 (K stops):** K stops = K+1 edges. Run Bellman-Ford for K+1 rounds — and relax off a *snapshot* of the previous round's distances so one pass can't use two new edges.

Warm-up note

These have no Easy LeetCode warm-up — the framing is the lift. The honest warm-up: solve LC 743 (Dijkstra) cold first, then read the Bellman-Ford code above and convince yourself why "relax all edges $V-1$ times" is correct. Floyd-Warshall is the shortest of the three; type it from memory once.

Practice (solve in this order — each unlocks the next)

Step	#	Problem	Difficult
1	787	Cheapest Flights Within K Stops — Bellman-Ford with $K+1$ rounds; canonical limited-edge	Medium
2	1334	Find the City With the Smallest Number of Neighbors at a Threshold — textbook Floyd-Warshall	Medium
3	399	Evaluate Division — Floyd-Warshall on ratios (multiply not add), or DFS; both work	Medium

Pattern 7 — MST — Minimum Spanning Tree (Kruskal + Prim)

One-liner: Connect all V vertices with $V-1$ edges of minimum total weight. Kruskal: sort edges, add each if it doesn't form a cycle (DSU). Prim: Dijkstra-like — grow the tree by the cheapest frontier edge.

Mental model

Kruskal: Sort all edges by weight. Iterate; for each (u, v, w) , if $\text{find}(u) \neq \text{find}(v)$, union them and add w to the cost. Stop after $V-1$ successful unions. $O(E \log E)$. Sparse graphs win.

Prim: Start from any vertex. Maintain a min-heap of edges crossing the cut (tree vs non-tree). Pop the cheapest; if its endpoint is non-tree, add it. Repeat until V nodes are in the tree. $O((V+E) \log V)$. Dense graphs win.

Why greedy works: Cut property — for any partition of V , the minimum-weight edge crossing the cut MUST be in some MST. Kruskal and Prim are systematic ways to apply this.

Invariant: At every step the edges chosen so far are a subset of SOME MST. Never undo a choice.

Spot signals

- "Min cost to connect all points / cities / nodes"
- "Min Cost to Connect All Points" (LC 1584) — the canonical problem
- "Optimize water distribution" / "network connection / cable layout cost"
- Any "spanning tree", "spanning forest", or "connect everything with minimum cost" framing.

Walkthrough — LC 1584 Min Cost to Connect All Points (Kruskal on a complete graph)

Build all $C(N,2)$ edges with Manhattan distance, sort, run Kruskal. The DSU class is reused verbatim from Pattern 4.


```

public int minCostConnectPoints(int[][] points) {
    int n = points.length;
    List<int[]> edges = new ArrayList<>();          // {weight, i, j}
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int w = Math.abs(points[i][0] - points[j][0])
                + Math.abs(points[i][1] - points[j][1]);
            edges.add(new int[]{w, i, j});
        }
    }
    edges.sort((a, b) -> Integer.compare(a[0], b[0]));
    DSU dsu = new DSU(n);                        // from Pattern 4
    int cost = 0, used = 0;
    for (int[] e : edges) {
        if (dsu.union(e[1], e[2])) {
            cost += e[0];
            if (++used == n - 1) break;           // tree complete – stop early
        }
    }
    return cost;
}

```

Common traps

- **Not stopping early.** After $V-1$ successful unions you have the MST — the rest of the sorted edges are wasted work.
- **Confusing MST with the shortest-path tree.** They are NOT the same. SPT minimises distance from a source; MST minimises total weight. They can be different trees.
- **Forgetting that an MST may not be unique.** Two edges with equal weight → there can be multiple valid MSTs with the same total cost.
- **Disconnected graphs.** If the graph isn't connected there's no spanning tree — you'd build a minimum spanning FOREST instead. Detect it when $\text{used} < V - 1$ after exhausting edges.

Warm-up note

MST has no Easy LeetCode warm-up. The hardest part is owning the DSU class from Pattern 4 — if you can write DSU cold, Kruskal is ~8 lines of glue on top. Do Pattern 4's warm-up (LC 547, 684) first, then jump straight to LC 1584.

Practice (solve in this order — each unlocks the next)

Step	#	Problem	Difficulty
1	1584	Min Cost to Connect All Points — canonical MST; Kruskal on Manhattan-distance edges	Medium
2	1489	Find Critical and Pseudo-Critical Edges in MST — Kruskal with one edge forced / omitted	Hard
3	1579	Remove Max Edges to Keep Graph Fully Traversable — dual-colour DSU; FAANG-grade variant	Hard

Kruskal + DSU covers the entire MST surface. LC 1584 is the canonical warm-up; LC 1489 and LC 1579 are the FAANG-grade extensions. Premium MST variants (LC 1135 Connecting Cities, LC 1168 Optimize Water Distribution) reuse the same template.

Pattern 8 — Grid as graph (flood fill, multi-source BFS, boundary expansion)

One-liner: A 2D grid IS a graph — every cell connects to its 4 (or 8) neighbours. Compute neighbours on the fly with direction deltas; never build an adjacency list.

Mental model

Grid problems are graph problems with an implicit adjacency list. The DFS / BFS / Dijkstra skeletons from Patterns 1, 2, 5 all work — just swap for `(int v : graph.get(u))` for `for (int[] d : dirs)`.

Direction deltas: `int[][] dirs = {{-1,0},{1,0},{0,-1},{0,1}}` for 4-connected; add the four diagonals for 8. `int nr = r + d[0], nc = c + d[1]`; keeps the inner loop short and bug-free.

Boundary-expansion trick (LC 130, 417): Instead of asking "which cells are TRAPPED?", ask "which cells reach the BORDER?" Start BFS / DFS from the border inward, mark survivors, flip the rest.

Bounds + visited check: Two guards at the top of every recursive call — if `(r < 0 || r >= R || c < 0 || c >= C)` return; AND if `(visited[r][c] /* or wrong colour */)` return;. Both are needed.

Spot signals

- "Number of islands" / "max area of island" / "island perimeter"
- "Flood fill" / "surrounded regions" / "pacific atlantic water flow"
- "Shortest path in a binary matrix" / "knight's shortest path"
- "Word search" / "path sum in grid" — DFS with backtracking on cells.
- "Rotting oranges" / "01 matrix" / "walls and gates" — multi-source BFS giveaway.

Walkthrough — LC 1091 Shortest Path in Binary Matrix (8-directional BFS)

8-directional BFS from (0,0) to (n-1,n-1). Mark visited on enqueue. Track distance in the queue tuple.

```
public int shortestPathBinaryMatrix(int[][] grid) {
    int n = grid.length;
    if (grid[0][0] == 1 || grid[n - 1][n - 1] == 1) return -1;
    int[][] dirs = {{-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},{1,1}};
    Deque<int[]> q = new ArrayDeque<>();
    q.offer(new int[]{0, 0, 1});           // {r, c, dist}
    grid[0][0] = 1;                       // mark visited
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        int r = cur[0], c = cur[1], d = cur[2];
        if (r == n - 1 && c == n - 1) return d;
        for (int[] dir : dirs) {
            int nr = r + dir[0], nc = c + dir[1];
            if (nr >= 0 && nr < n && nc >= 0 && nc < n && grid[nr][nc] == 0) {
                grid[nr][nc] = 1;           // mark on enqueue
                q.offer(new int[]{nr, nc, d + 1});
            }
        }
    }
    return -1;
}
```

Walkthrough — LC 417 Pacific Atlantic (boundary-expansion trick)

Don't ask "which cells flow to both?" — ask "which cells does the PACIFIC reach going uphill?" and the same for the Atlantic. Intersect the two reachable sets.

```
private static final int[][] D = {{-1,0},{1,0},{0,-1},{0,1}};

public List<List<Integer>> pacificAtlantic(int[][] heights) {
    int R = heights.length, C = heights[0].length;
    boolean[][] pac = new boolean[R][C], atl = new boolean[R][C];
    for (int r = 0; r < R; r++) {
        dfs(heights, r, 0, pac, Integer.MIN_VALUE, R, C);
        dfs(heights, r, C - 1, atl, Integer.MIN_VALUE, R, C);
    }
    for (int c = 0; c < C; c++) {
        dfs(heights, 0, c, pac, Integer.MIN_VALUE, R, C);
        dfs(heights, R - 1, c, atl, Integer.MIN_VALUE, R, C);
    }
    List<List<Integer>> res = new ArrayList<>();
    for (int r = 0; r < R; r++)
        for (int c = 0; c < C; c++)
            if (pac[r][c] && atl[r][c]) res.add(List.of(r, c));
    return res;
}

private void dfs(int[][] h, int r, int c, boolean[][] vis, int prev, int R, int C) {
    if (r < 0 || r >= R || c < 0 || c >= C) return;
    if (vis[r][c] || h[r][c] < prev) return; // uphill only
    vis[r][c] = true;
    for (int[] d : D) dfs(h, r + d[0], c + d[1], vis, h[r][c], R, C);
}
```

Common traps

- **Forgetting one of the two guards (bounds OR visited).** Crashes on edge cells / loops forever on cycles. Always both.
- **Mutating the grid for visited tracking when the problem says "do not modify input."** Use a separate boolean[][] visited.
- **Iterating 4 directions when the problem allows 8 (LC 1091).** Re-read the move set.
- **Word search backtracking: forgetting to UNMARK the cell on the way back up.** The path muscle from Pattern 1 carries over: mark, recurse, UNMARK.
- **Multi-source BFS with sources enqueued one-at-a-time inside the loop.** Push ALL sources BEFORE the while loop — the same wave starts everywhere.

Warm-up first — build the mechanics

Solve this cold before LC 1091. LC 463 has zero recursion / queue — just count exposed cell edges. It locks the "bounds check + neighbour delta" rhythm without any traversal on top.

#	Problem	Difficulty
463	Island Perimeter — single pass counting exposed edges; pure direction-delta practice	Easy

Practice (solve in this order — each unlocks the next)

Step	#	Problem	Difficulty
1	79	Word Search — DFS with backtracking on a grid; canonical mark / recurse / unmark	Medium
2	417	Pacific Atlantic Water Flow — boundary-expansion trick; Google bar-raiser	Medium
3	1091	Shortest Path in Binary Matrix — 8-directional BFS; mandatory FAANG grid problem	Medium
4	130	Surrounded Regions — reverse flood fill from the border; reuses Pattern 1 muscle	Medium

Pattern 9 — Bipartite check (2-coloring via BFS / DFS)

One-liner: A graph is bipartite iff you can 2-color it so that no edge connects same-colored nodes. BFS / DFS while assigning alternating colors; a same-color neighbour means NOT bipartite.

Mental model

Start any uncolored node, assign color 0, then BFS / DFS. Every neighbour gets the OPPOSITE color. If you ever try to color a neighbour and it already has the SAME color, the graph is not bipartite. Repeat for every component.

Why two colors suffice: Bipartite = no odd-length cycles. If every cycle has even length, alternating colors closes consistently around it. An odd cycle forces a contradiction at the closing edge.

Color scheme: `int[] color = new int[n]; Arrays.fill(color, -1);` — -1 = uncolored, 0/1 the two sides. Initialize the start node to 0, then `color[v] = 1 - color[u]` on every edge expansion.

Coverage of all components: `for (int v = 0; v < n; v++) if (color[v] == -1) bfs(v);`. A single BFS without this outer loop misses disconnected pieces and fails LC 785's hidden tests.

Spot signals

- "Can the nodes be split into two groups so that every edge goes BETWEEN groups?"
- "Possible bipartition" / "two teams" / "two halves" / "split into two sets"
- "Is this graph 2-colorable / two-colorable?"
- Anything that smells like "avoid same-side neighbours" or "no conflicts within a team".

Walkthrough — LC 785 Is Graph Bipartite (BFS 2-coloring)

Iterate every node. For each uncolored node, run BFS assigning alternating colors. If a same-color edge appears, return false.

```

public boolean isBipartite(int[][] graph) {
    int n = graph.length;
    int[] color = new int[n];
    Arrays.fill(color, -1); // -1 = uncolored
    for (int s = 0; s < n; s++) { // cover every component
        if (color[s] != -1) continue;
        color[s] = 0;
        Deque<Integer> q = new ArrayDeque<>();
        q.offer(s);
        while (!q.isEmpty()) {
            int u = q.poll();
            for (int v : graph[u]) {
                if (color[v] == -1) {
                    color[v] = 1 - color[u]; // opposite color
                    q.offer(v);
                } else if (color[v] == color[u]) { // same-color edge -> odd cycle
                    return false;
                }
            }
        }
    }
    return true;
}

```

Common traps

- **Forgetting the outer for-loop over components.** LC 785's hidden tests always include a disconnected case.
- **Only checking the FIRST neighbour relation.** Every edge needs the same-color check, including back-edges.
- **Using a single boolean[] visited instead of int[] color.** You need THREE states (uncolored / 0 / 1), not two.
- **LC 886: the input is dislike PAIRS, not the adjacency list.** Build adj from the pairs FIRST, then run the same algorithm.
- **Deep recursive DFS on N up to 10⁴.** Java can hit StackOverflowError on a long component. Prefer iterative BFS (as above) or a big-stack thread.

Warm-up first — build the mechanics

Solve LC 785 cold before moving on. It is THE canonical bipartite problem, and the BFS template above is reused unchanged on every other problem in this pattern.

#	Problem	Difficulty
785	Is Graph Bipartite? — canonical 2-coloring BFS; the template every other problem reuses	Medium

Practice (solve in this order — each unlocks the next)

Step	#	Problem	Difficulty
1	886	Possible Bipartition — same algorithm; the twist is building adj from dislike pairs first	Medium
2	1042	Flower Planting With No Adjacent — relaxed 4-coloring; greedy smallest unused color	Medium

3	1129	Shortest Path with Alternating Colors — bipartite-flavored BFS with color state in the queue	Medium
---	------	--	--------

Cross-pattern traps (read once before any interview)

Trap	Patterns at risk	How to dodge
Marking visited on dequeue (BFS) or after recursing (DFS)	1, 2, 8	Mark when you ENQUEUE / before you recurse. A node never enters the frontier twice.
Single boolean[] visited for cycle detection in a directed graph	1, 3	Use 3-colour (WHITE/GRAY/BLACK) or an onStack[] flag. visited[] alone misses back-edges.
Building the topo-sort graph in the wrong direction	3	Read the input twice. [course, prereq] means prereq → course. Test on a 3-node example.
DSU without BOTH path compression AND union by rank	4, 7	Memorise the class. Both optimisations are mandatory for amortised O(1).
Running Dijkstra on a graph with negative weights	5	Check the weight range FIRST. Negative → Bellman-Ford. Non-negative → Dijkstra.
Floyd-Warshall with k NOT in the outermost loop	6	Memorise: for k → for i → for j. That ordering is the correctness proof.
Integer overflow when adding to an "infinity" sentinel	5, 6	Relax only from finite nodes, use long, or a 1e9 sentinel — never MAX_VALUE you add to.
(a,b) -> a[0]-b[0] comparator overflow	5, 7	Use Integer.compare(a[0], b[0]) for any heap/sort of large or signed values.
Mixing up MST with the shortest-path tree	5, 7	MST minimises TOTAL weight; SPT minimises distance FROM A SOURCE. Different trees.
Deep recursion blowing the JVM stack	1, 3, 8, 9	No setrecursionlimit in Java — StackOverflowError. Go iterative or use a big-stack thread.
Grid: forgetting one of bounds / visited check	8	Two guards — out-of-bounds AND already-visited / wrong-colour. Both, every time, at the top.

Pre-interview revision drill (20 minutes)

Run this the night before:

- 30 seconds per pattern** — name the invariant out loud (all 9).
- 3 minutes per pattern** — write the skeleton from memory (no IDE). Especially Dijkstra and DSU — those should flow.
- 5 minutes** — walk through LC 207 (Course Schedule) Kahn's BFS. If you can argue why $\text{idx} < V$ means a cycle, your topo sort is solid.
- 5 minutes** — walk through LC 743 (Network Delay) Dijkstra. If you can explain why the stale-entry skip is correct, your Dijkstra is solid.
- 2 minutes** — name the #1 trap in each pattern (the cross-pattern table above).

If you fumble any of these → don't open new problems. Re-do the warm-up + canonical problem for that pattern.

7-day study plan

Day	Focus	Problems	Notes
1	Pattern 1 (DFS) + Pattern 8 warm-up	733, 200, 695, 841, 463	Foundation day. By end of day, the DFS skeleton should flow without thinking.
2	Pattern 2 (BFS) + finish Pattern 8	1971, 994, 102, 1091, 79	Wave thinking + grid mastery. LC 79 (Word Search) backtracking deserves extra time.
3	Pattern 3 (Topo Sort)	207, 210, 802, 310	All 4 in one day. Once you've done 207 + 210, the others are remixes.
4	Pattern 4 (Union-Find)	547, 684, 990, 721	Memorise the DSU class cold first. LC 721 (Accounts Merge) is the messy real-world test.
5	Pattern 5 (Dijkstra)	743, 1631, 778, 1514	Bar-raiser day. LC 778 deserves a re-solve next morning. LC 1514 swaps the relaxation sign.
6	Pattern 6 (Bellman-Ford + Floyd)	787, 1334, 399	Conceptually heavy, light on code. Type Floyd-Warshall from memory before opening any problem.
7	Pattern 7 (MST) + Pattern 9 (Bipartite) + revision	1584, 1489, 130, 417, 785, 886	MST = DSU + glue. Bipartite = BFS + alternating-color array. Then run the revision drill.

Day-plan total: 32 anchor problems across 7 days (the spine). The full sheet ships 36 — the extra reps (LC 1579 plus the deeper practice items) are stretch sets once the spine flows. Easy = warm-up; Medium = canonical; Hard = boss. If a problem takes > 30 minutes without progress, peek at the editorial, understand it, then write the code from scratch the next day.

The big idea

Graph looks like 30 different algorithms. It's 9 patterns × 3-5 reps each. The people who clear FAANG don't memorise 30 algorithm names — they own 9 templates cold and recognise which fits in 10 seconds.

Do the 36 must-do problems. Internalise the invariants. Then every future graph problem is just a remix.

Java note: This edition ships every template in idiomatic Java so there's nothing left to "translate in your head" mid-interview. The patterns travel — but the heap, the deque, the hash map, the overflow guards, and the recursion-depth fix are all Java-native here. Own these and you write them cold under pressure.

Java edition derived from the original "Graph Mastery" sheet by @rathoreadiitya. Patterns and problem set preserved; all code and language notes re-authored for Java.